



UNIVERSIDAD
DE LA FRONTERA

Análisis de seguridad en cadena de producción de software

Estudiantes: Daniela Díaz
David Millar

Docente: Mg. Pablo Valenzuela

Asignatura: Producción de Software (EMI305-1)

Fecha: 25 de mayo del 2026



Índice de contenidos

1. Introducción
 - a. Contexto
 - b. Alcance y objetivos
2. Desarrollo
 - a. Hallazgos
 - b. Categorización de riesgos
 - c. Mitigaciones
3. Cierre

Introducción

Contexto

Express es un **framework web minimalista para Node.js**, usado para **construir APIs, aplicaciones web y middleware HTTP**.

Desde sus primeras versiones, se consolidó como **una de las herramientas más influyentes del ecosistema JavaScript del lado del servidor** y, en la actualidad, sigue teniendo una presencia importante en proyectos backend, tanto por su madurez como por su amplia comunidad.



*Figura 1. Logotipo del proyecto
(ExpressJS, 2021)*

Introducción

Contexto

Los repositorios analizados pertenecen a su ecosistema:

- Algunos son el **núcleo del framework**
- Otros son **middlewares** usados para manejar sesiones, cookies, CORS, compresión, formularios, logs y documentación.
- Se incluye el **sitio web oficial del proyecto** (expressjs.com)



*Figura 1. Logotipo del proyecto
(ExpressJS, 2021)*

Repositorio	Contenido	Rol dentro del ecosistema
express	Framework web principal para Node.js.	Núcleo para crear rutas, middlewares y aplicaciones HTTP.
multer	Middleware para manejar <code>multipart/form-data</code> .	Se usa para subida de archivos y formularios con archivos.
morgan	Middleware de logging HTTP.	Registra peticiones para auditoría, debug y monitoreo.
session	Middleware de sesiones para Express.	Maneja estado de usuario mediante cookies y almacenamiento de sesión.
cors	Middleware para configurar CORS.	Controla que orígenes pueden consumir recursos HTTP.
body-parser	Middleware para parsear cuerpos de request.	Procesa JSON, texto, raw y formularios urlencoded.
expressjs.com	Sitio web oficial de Express.	Publica documentación y contenido del proyecto.
compression	Middleware de compresión HTTP.	Reduce tamaño de respuestas con gzip/brotli cuando aplica.
csurf	Middleware historico de protección CSRF.	Ayuda a mitigar ataques Cross-Site Request Forgery.
cookie-parser	Middleware para parsear cookies.	Lee y firma cookies recibidas en requests.

Tabla 1. Repositorios de ExpressJS analizados, sus contenidos y su rol.

Introducción

Alcance y objetivos

Se plantea como **alcance de la actividad**:

- Revisar dependencias y código fuente de repositorios
- Revisar evidencia generada por herramientas de seguridad
- Clasificar hallazgos por vector de ataque
- Proponer acciones de gestión de riesgo



*Figura 1. Logotipo del proyecto
(ExpressJS, 2021)*

Introducción

Alcance y objetivos

En relación a lo anterior, **se busca:**

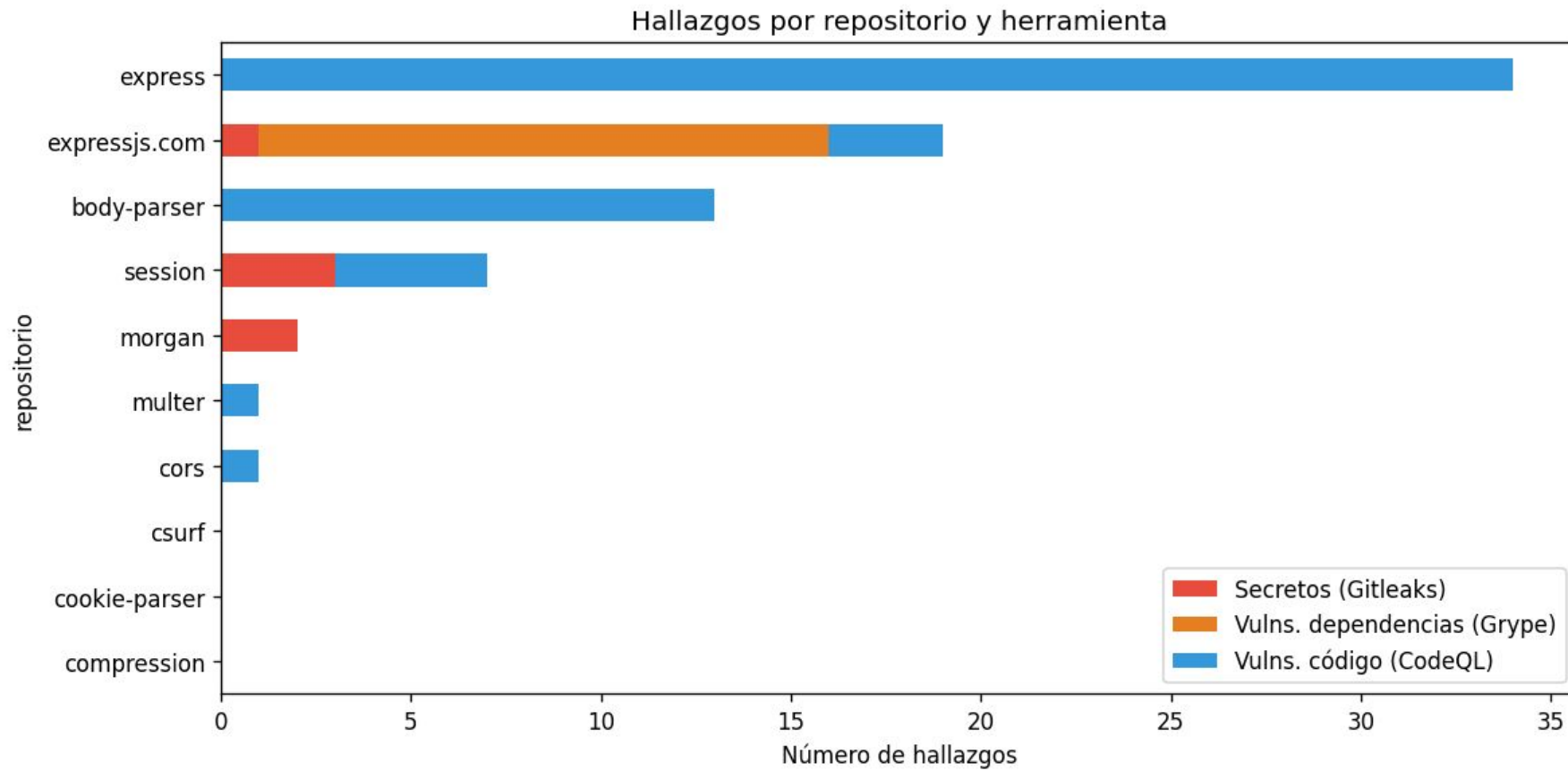
- Identificar hallazgos relevantes
- Separar vulnerabilidades reales de falsos positivos o fixtures
- Priorizar según severidad, exposición, explotabilidad, impacto y evidencia
- Proponer mitigaciones correspondientes de carácter preventivo y correctivo



*Figura 1. Logotipo del proyecto
(ExpressJS, 2021)*

Desarrollo

Hallazgos



Conozco	Verifico	Evidencio	Decido y actuo
5 claves privadas RSA expuestas que podrían transparentar información encriptada. 1 API key de Algolia en expressjs.com, permite el acceso no autorizado al servicio.	Se evidencia que las claves RSA expuestas corresponden a archivos generados con propósitos de pruebas unitarias/de integración. La API key de Algolia tiene un formato aceptado por el servicio, y es utilizada por el código.	Ambos hallazgos fueron encontrados por Gitleaks, cuyos resultados se almacenan en los archivos: - morgan-gitleaks.json - session-gitleaks.json - expressjs.com-gitleaks.json La API Key se encuentra expuesta en <code>_data/docsearch.yml</code> Las claves privadas se encuentran en directorios fixtures de ambos repositorios (<code>test/fixtures/server.key</code>)	Se debe: - Rotar API key de Algolia. - Reemplazar private keys por generación dinámica en tests. - Limpiar historial con <code>git filter-repo</code>. - Añadir Gitleaks en CI para prevenir futuras fugas.

Tabla 2. Hallazgos de secretos expuestos, vector humano

Conozco	Verifico	Evidencio	Decido y actuo
Grype detectó 15 vulnerabilidades activas en expressjs.com: 7 High 7 Medium 1 Low Paquetes como Vite 7.3.1, Picomatch 2.3.1/4.0.3, Astro 6.1.2, etc. Ninguna dependencia ha sido actualizada.	Analizando las dependencias que se detectaron con vulnerabilidades, encontramos casos como los siguientes: Vite 7.3.1 tiene 2 vulnerabilidades High: lectura de archivos arbitrarios vía WebSocket del dev server y bypass de server.fs.deny mediante queries en la URL, permitiendo a un atacante robar código fuente, credenciales y archivos sensibles. defu 6.1.4 tiene 1 vulnerabilidad High: prototype pollution vía __proto__ en el argumento defaults, permitiendo a un atacante inyectar propiedades en todos los objetos del programa, pudiendo escalar a XSS o ejecución remota de código, generando una Weird Machine.	Las alertas generadas por Grype se evidenciaron en el repositorio expressjs.com se registraron en el documento expressjs.com-grype.json. Conteniendo las 15 vulnerabilidades con ID, severidad, CVSS y versión con fix disponible.	Se debe: - Auditar dependencias para identificar y eliminar las que no se usan directamente. - Actualizar las dependencias a las versiones que contienen los fixes, usando npm update u overrides para forzar versiones seguras en transitivas. - Configurar Dependabot (o una herramienta similar) para automatizar las actualizaciones y mantener las dependencias del producto al día.

Tabla 3. Hallazgos de dependencias vulnerables, vector de dependencias y código

Conozco	Verifico	Evidencio	Decido y actuo
CodeQL encontró 11 alertas XSS activas en 3 repositorios: expressjs.com (3): SidebarVersionManager.ts usa innerHTML con datos sin escapar, y orama-documents.mjs tiene una sanitización incompleta. Afectan al sitio en producción. body-parser (8): XSS reflejado y por excepción en archivos de test. No afectan producción directamente. session (2): XSS reflejado en archivos de test. No afectan producción directamente.	expressjs.com usa innerHTML con datos que podrían contener HTML malicioso. Está en producción, el componente renderiza versiones en el sidebar del sitio. body-parser y session, las alertas están en archivos de test (test/json.js, test/session.js), no en código desplegado. orama-documents.mjs sanitización incompleta que permite inyectar <script>. El archivo procesa todo el contenido de la documentación y se ejecuta en CI/CD (sync-orama.yml) para indexar la búsqueda del sitio.	Las 11 alertas están registradas en: - body-parser-codeql.json - session-codeql.json - expressjs.com-codeql.json cada una con la regla, el archivo y la línea exacta del código vulnerable.	Se debe: - Corregir SidebarVersionManager.ts reemplazando innerHTML por textContent. - Corregir orama-documents.mjs usando una librería de escape en vez de regex. - Los tests se dejan como prioridad baja, pero se establece como política no usar innerHTML ni en tests. - Agregar regla de ESLint no-unsafe-innerHTML para prevenir futuros XSS.

Tabla 4. Hallazgos de code smells, vector de dependencias y código

Conozco	Verifico	Evidencio	Decido y actúo
Se hallan 8 puntos que afectan a la capa de seguridad de las aplicaciones: cifrado, control de acceso, autenticación y registros. express (4): 3 cookies sin Secure y contraseñas en logs cors (1): CORS abierto a cualquier origen session (1): hash débil para sessionId body-parser (2): datos de usuario en logs sin sanitizar	Cada alerta fue confirmada revisando el código en la línea exacta que CodeQL señaló: express: Cookies viajan en texto plano si la conexión no es HTTPS; La contraseña se pasa directamente a console.log(), sin ningún cifrado previo. cors: El middleware está configurado para reflejar cualquier origen enviado, sin validación. session: El hash del sessionId usa un algoritmo considerado débil hoy en día, como SHA1 o MD5, que puede romperse con fuerza bruta moderna. body-parser: Las funciones toman datos del body de la petición y los concatenan directamente al mensaje de log, sin escaparlos ni validarlos.	Las alertas están en: data/results/express-codeql.json (cookies sin Secure + password en logs) cors-codeql.json (CORS permisivo) session-codeql.json (crypto débil) body-parser-codeql.json (log injection), cada una con regla, archivo, línea y descripción del problema.	Se debe: 1. Agregar Secure y HttpOnly a cookies, evita interceptación. 2. Eliminar console.log(password), protege credenciales. 3. Restringir CORS a orígenes específicos, evita exfiltración. 4. Reemplazar crypto débil por SHA-256, protege sesiones. 5. Sanitizar entradas en logs, evita log injection.

Tabla 5. Hallazgos de malas prácticas de configuración, vector de dependencias y código

Desarrollo

Categorización de riesgos

ID	Vulnerabilidad	Riesgo	Probabilidad	Impacto
V01	Dependencias vulnerables	Medio	Alta	Medio
V02	Code smells de seguridad	Alto	Alta	Alto
V03	Secretos expuestos	Alto	Baja	Alto
V04	Malas prácticas de configuración	Alto	Alta	Alto
V05	Malas prácticas en pipelines	Bajo	Baja	Medio

Tabla 6. Categorización de grupos de riesgos

Desarrollo

Mitigaciones (Dependencias vulnerables)

ID	ID Vulnerabilidad	Descripción	Naturaleza	Impacto esperado
M01	V01	Actualización de dependencias (Vite)	Correctiva	Vite se actualiza a su versión 7.3.2, solucionando 3 vulnerabilidades, incluyendo “arbitrary file read CVSS 8.2”
M02	V01	Actualización de dependencias (Picomatch, defu, devalue)	Correctiva	Las dependencias se actualizan a su versión más reciente, solucionando vulnerabilidades graves (ReDoS, prototype pollution, DoS)
M04	V01	Configuración de Dependabot	Preventiva	Se obtiene un reporte semanal acerca del estado de las dependencias utilizadas en los repositorios

Desarrollo

Mitigaciones (Code smells)

ID	ID Vulnerabilidad	Descripción	Naturaleza	Impacto esperado
M04	V02	Corrección de vulnerabilidad XSS	Correctiva	Se reemplaza código sensible en SidebarVersionManager.ts y orama-documents.mjs, mitigando la posibilidad de ataques XSS.
M05	V02	Auditoría de pruebas XSS	Preventiva	Las alertas están en tests, no producción. En ceremonias de revisión de código se verifica que los patrones no pasen a código productivo.
M06	V02	Reglas de linting contra XSS	Preventiva	Se utilizan complementos como eslint-plugin-security o no-unsafe-innerHTML para prevenir reintroducción.

Desarrollo

Mitigaciones (Secretos expuestos)

ID	ID Vulnerabilidad	Descripción	Naturaleza	Impacto esperado
M07	V03	Parametrización de API key por medio de secretos del repositorio	Preventiva	La API key deja de estar expuesta como valor plano en el repositorio
M08	V03	Eliminación de secretos del repositorio	Correctiva	Los secretos ya no serán visibles al explorar los archivos y commits del repositorio
M09	V03	Generación de claves dinámicas en testing	Correctiva	La naturaleza de las claves no estará expuesta directamente en el repositorio
M10	V03	Pipeline de verificación de secretos	Preventiva	Si un secreto es expuesto en alguna ceremonia de integración, se alerta y se cancela de manera inmediata

Desarrollo

Mitigaciones (Malas prácticas en configuración)

ID	ID Vulnerabilidad	Descripción	Naturaleza	Impacto esperado
M11	V04	Restringir CORS	Correctiva	En cors/lib/index.js, se cambia de origen permisivo a lista blanca de dominios
M12	V04	Migración a algoritmo de encriptado más seguro	Correctiv	Se reemplaza algoritmo débil por SHA-256 o mejor en session/index.js:625
M13	V04	Secure flag en cookies (De ejemplo)	Correctiva	Se añade { secure: true, httpOnly: true } a las cookies en los 3 ejemplos de express
M14	V04	Logging enmascarado	Preventiva	Los logs relacionados a claves e PII estarán enmascarados desde el lado del servidor.

Desarrollo

Mitigaciones (Malas prácticas en pipelines)

ID	ID Vulnerabilidad	Descripción	Naturaleza	Impacto esperado
M15	V05	Auditoría de pipelines	Correctiva	Por cada ciclo iterativo, se obtiene una revisión de las pipelines de los repositorios, indicando vulnerabilidades incorporadas por iteración

Cierre

Análisis de seguridad en cadena de producción de software

En relación a las vulnerabilidades conocidas, sus riesgos
y sus mitigaciones:

- Se opta por priorizar las mitigaciones correctivas ante todo, enfatizando los esfuerzos en los **code smells de seguridad y en las malas prácticas de configuración**
- Las mitigaciones preventivas son cruciales frente a futuros incidentes, permitiendo ahorrar en potenciales costos correctivos y manteniéndose en línea con estándares de seguridad en la industria (o normativos)



UNIVERSIDAD
DE LA FRONTERA

Análisis de seguridad en cadena de producción de software

Estudiantes: Daniela Díaz
David Millar

Docente: Mg. Pablo Valenzuela

Asignatura: Producción de Software (EMI305-1)

Fecha: 25 de mayo del 2026

